Paper Summaries

Filippo Nardon, Giovanni Maria Zanchetta

November 7, 2026

Contents

1. Hallem, Chelf, Xie, Engler — *A System and Language for Building System-Specific, Static Analyses* (PLDI 2002) 2
Goal and philosophy . . . 2
Metal: checkers as state machines . . . 2
Intraprocedural algorithm . . . 2
Interprocedural algorithm . . . 3
Reducing false positives . . . 3
Ranking . . . 3
Takeaways . . . 3

2. Das, Lerner, Seigle — *ESP: Path-Sensitive Program Verification in Polynomial Time* (PLDI 2002) 3
Goal . . . 3
Motivating example . . . 4
Property analysis framework . . . 4
Instantiation and complexity . . . 4
The ESP system: multiple values and aliasing . . . 4
Case study . . . 5
Takeaways . . . 5

3. Fink, Yahav, Dor, Ramalingam, Geay — *Effective Typestate Verification in the Presence of Aliasing* (TOSEM 2008) 5
Goal . . . 5
Setting . . . 5
Parametric abstract domain . . . 5
The verifiers (increasing cost and precision) . . . 6
Further techniques . . . 6
Implementation and results . . . 6
Takeaways . . . 7

Comparison 7

These summaries assume the background of *Static Program Analysis* (Møller & Schwartzbach, `spa.pdf`): lattices and fixed points, monotone frameworks, MFP vs. MOP, path sensitivity, interprocedural analysis (call strings and the functional approach), IFDS/IDE, points-to analysis (allocation-site abstraction, Andersen, Steensgaard), and abstract interpretation. Only the ideas that go beyond the book are explained in detail; references like "SPA §8.4" point to the relevant book section.

All three papers deal with the same underlying problem: checking that a program uses an API according to a **protocol** described by a finite state machine (open before read, lock before unlock, no use after free, . . .). This is known as **typestate** checking (Strom & Yemini, 1986): besides its type, each object carries a state of an automaton, operations on the object are transitions, and reaching the distinguished `err` state is a bug.

1. Hallem, Chelf, Xie, Engler — *A System and Language for Building System-Specific, Static Analyses* (PLDI 2002)

Goal and philosophy

The paper describes the "metacompilation" approach: **metal**, a language in which programmers write small, rule-specific checkers, and **xgcc**, the engine that runs those checkers over C code with a context-sensitive, interprocedural analysis. The explicit goal is to *find as many bugs as possible*, not to prove correctness: the analysis is **deliberately unsound** (it gives up both soundness and completeness in the SPA §1.2 sense) and relies on suppression and ranking to keep false positives manageable. The system found thousands of bugs in Linux and OpenBSD.

Metal: checkers as state machines

A checker is a collection of state machines (SMs):

- A **global state variable** (one instance, e.g. "interrupts disabled").
- A **variable-specific state variable**, with one instance *per tracked program object* (e.g. "pointer `p` is freed"). Instances are created and destroyed dynamically, so the number of SMs grows and shrinks during the analysis.
- The state of a checker is a **set of tuples** (global value, one variable-specific instance), each tuple being an independent SM.

Transitions are triggered by **patterns**: fragments of C code with typed *hole variables* (e.g. `any_pointer v`, `any_fn_call`, `any_arguments`) that are matched against ASTs. Example, the free checker:

```
state decl any_pointer v;
start:   { kfree(v) } ==> v.freed;
v.freed: { *v }       ==> v.stop, { err("use after free"); }
       | { kfree(v) } ==> v.stop, { err("double free"); }
```

Extensions to the plain SM model:

- **Path-specific transitions**: at a branch, a different destination state for the true and false edge (e.g. `trylock(l) != 0` ⇒ `locked` on true, `stop` on false).
- **C code actions** attached to transitions: arbitrary computations, used for error reporting, counting (for statistical rule inference), or extra per-instance data (e.g. a lock depth counter, which makes the state space infinite).
- **Callouts** (`${ ... }`): C boolean expressions used as patterns.
- **Composition**: checkers can annotate ASTs for later checkers (e.g. mark `panic()` calls so paths through them are cut).
- Special patterns such as `$end_of_path$`.

Intraprocedural algorithm

xgcc does *not* use a classic worklist solver. It performs a **depth-first traversal of the CFG, one path at a time**, applying the checker to each AST node in execution order. To avoid exponential path enumeration it uses **block-level caching**: each basic block stores a summary of the state tuples that have already reached it; when a path reaches a block with only already-seen tuples, the path is cut. The result is, in the authors' words, similar to a fixed point of a dataflow analysis where the lattice element at a block is the set of tuples seen there (a powerset lattice, SPA ch. 4–5).

Two restrictions make this caching sound w.r.t. the checker semantics:

1. **Determinism**: applying the checker to the same program point in the same tuple always gives the same result.
2. **Independence**: transitions of the instance attached to one object cannot depend on other variable-specific instances (they may depend on the global state). This lets tuples be handled separately — the same "one fact at a time" idea behind IFDS (SPA §9.4) — making the cost linear rather than exponential in the number of tracked objects.

Block summaries contain two kinds of edges: **transition edges** $(s, v{:}t \to v_s) \to (s', v{:}t \to v'_s)$ and **add edges** $(s, v{:}t \to \mathit{unknown}) \to (s', v{:}t \to v'_s)$ for instances created inside the block.

Interprocedural algorithm

The analysis runs **top-down** from callgraph roots, over a supergraph (as in RHS/IFDS). At each call:

- **Refine/restore**: state is moved from actuals to formals (and through `&x`, `*x`, `x.f`, `x->f`) when entering the callee, and back on return; caller-local state is saved and restored; file-scope variables are temporarily inactivated when crossing files.
- **Function summaries** (built from add and transition edges, propagated backwards along a path by a `relax` routine as **suffix summaries**) are memoized; on a cache hit the callee is not re-analyzed and its summary is applied, as in the functional approach to context sensitivity (SPA §8.4).
- Unlike RHS, the state space need not be finite or known in advance, because functions are only analyzed in states that actually reach them along interprocedurally valid paths.

Recursion is handled unsoundly (an incomplete summary is reused in recursive cycles), as is value flow.

Reducing false positives

- **Killing**: when a variable (or a sub-expression like `i` in `a[i]`) is redefined, the state attached to it is dropped. Described as the single most effective technique.
- **Synonyms**: after `q = p`, state changes on `p` are mirrored on `q` (a cheap, local stand-in for alias analysis).
- **False-path pruning**: a simple path-sensitive analysis (SPA ch. 7) tracks equalities and comparisons with constants, uses congruence closure to infer equalities between variables, and prunes branches whose condition is decided. Loops simply set modified variables to unknown.
- **Targeted suppression** of known idioms inside the checker, and **history** (remembering past false positives across versions).

Ranking

Reports are stratified by severity (e.g. `SECURITY` > `ERROR` > `MINOR`) and sorted by ease of inspection (distance in lines, number of conditionals, synonym chains, local before interprocedural). The novel part is **statistical ranking**: for a rule followed e times and violated c times ($n = e + c$), compute the z-statistic

$$z(n, e) = \frac{e/n - p_0}{\sqrt{p_0(1 - p_0)/n}}, \qquad p_0 = 0.5.$$

Rules that are almost always followed but occasionally violated rank highest; analysis mistakes that cause a burst of reports sink to the bottom.

Takeaways

- Protocol rules map naturally to SMs over syntactic patterns.
- Engine: path-by-path DFS + per-block caching of independent tuples + summaries ≈ an IFDS-like interprocedural analysis on a dynamically-grown domain.
- The system trades soundness for expressiveness and scale; precision comes from cheap heuristics (killing, synonyms, path pruning) and ranking rather than from heavy analyses.

2. Das, Lerner, Seigle — *ESP: Path-Sensitive Program Verification in Polynomial Time* (PLDI 2002)

Goal

Verify (soundly) that a large C program obeys a temporal safety property given as an FSM (e.g. `fopen`/`fprintf`/`fclose` on file handles). Full path sensitivity (SPA ch. 7) would be precise but exponential; a plain monotone-framework analysis (SPA ch. 5) that joins at merge points loses branch correlations and reports false errors. ESP proposes **property simulation**, a middle ground with polynomial complexity.

Motivating example

```
if (dump) f = fopen(...);    // Open
if (p) x = 0; else x = 1;
if (dump) fclose(f);         // Close
```

- *Dataflow analysis* joins after the first `if`, getting $\{\$uninit, Opened\}$ with no information on `dump`, so `fclose` may run in `$uninit` ⇒ **false error**.
- *Fully path-sensitive analysis* keeps all 4 paths (correlating `dump`, `p`, `x`) — correct but exponential in general.
- *Property simulation* keeps `dump` separate (because the FSM state differs on the two branches) but merges the `p`/`x` branches (they do not change the FSM state): 2 states, no false error.

Property analysis framework

Dataflow facts are sets of **symbolic states** $[\,as, es\,]$, where $as \subseteq D$ is a set of property (FSM) states and es is an *execution state* (an abstraction of the program store). The analysis is a standard worklist algorithm with three flow functions:

- F_{mrg} (merge nodes): union of incoming sets;
- F_{br} (branch nodes): uses a "theorem prover" on es to discard infeasible directions, otherwise adds the branch predicate to es;
- F_{oth} (other nodes): applies FSM transitions and updates es.

Each result is filtered by a **grouping function** α, and the choice of α yields three analyses:

α	behaviour	analysis
$\alpha_{cs}(ss) = ss$	never merge	fully path-sensitive
α_{df}	merge everything into one state (join all as and all es)	standard dataflow
α_{as}	merge the es of states **with the same property state**	property simulation

The heuristic behind α_{as}: a branch is relevant iff the FSM ends up in different states along its arms. This mirrors how careful programmers write code — they keep an implicit correlation between a flag and the protocol state, and ESP makes it explicit. In terms of SPA, it is a form of *relational/path-sensitive analysis* (§7.2) where the partitioning is by property state only (a finite set), and joins are applied within each partition.

Instantiation and complexity

ESP instantiates es with the **constant propagation lattice** (SPA §5.2): stores mapping variables to constants or $\top$; the "theorem prover" substitutes known values into conditions and simplifies them. Since each fact contains at most $|D|$ symbolic states and each can descend at most H times (finite height lattice, or widening — SPA ch. 6), intraprocedural complexity is $O(H\,|E|\,|D|\,(T + J + Q))$, i.e. $O(V^2|E||D|)$ for constant propagation.

Interprocedurally, property simulation is built on **RHS-style summaries** (SPA §8.4, §9.4): summaries map an input property state to output symbolic states, so the analysis is **context-sensitive in the property state** but **context-insensitive in the execution state** (execution states coming from different call sites are merged at function entry), which guarantees termination. Complexity: $O(V^2|D|(|E||D| + Calls\,|D|^2))$.

Property simulation can lose precision when a flag is copied *before* being used to guard any transition and the copies guard different transitions; the authors argue the common patterns (flag set together with the transition, flag passed to another function) are handled precisely.

The ESP system: multiple values and aliasing

Real programs have many stateful values (many file handles) flowing through pointers. ESP borrows metal's specification style (FSM + source patterns + value-creation patterns) but must stay **sound**, so it needs value-flow information. Key ideas:

1. **Split the problem in two**: a scalable flow-insensitive, context-sensitive points-to analysis (SPA ch. 11) builds a **value flow graph** (VFG) answering "may the value of e flow to e'?"; property simulation then uses the VFG instead of tracking aliasing itself.
2. **Track one stateful value at a time** (like a bit-vector analysis): branches affecting other values are merged away, amplifying the merge heuristic.

Pipeline: call graph and CFGs (indirect calls resolved by points-to) → VFG → abstract CFG with pattern nodes → **interface expressions** (inNodes/outNodes of each function, pruned by bottom-up slicing, plus mod sets) → **alias sets** (sets of expressions that may hold the tracked value, created where the value escapes, like an escape analysis) → interprocedural property simulation.

Because the VFG is only *may* information, when a pattern applies to an expression that may hold the tracked value *or* another value, ESP adds both the transition and the identity transition — i.e. a **weak update**.

Case study

gcc (SPEC95, 140 KLOC, 2149 functions, an SCC of 450+ functions): 15 output files are opened conditionally on uncorrelated flags, so path-sensitive methods would face 2^{15} combinations. ESP verified that all **646 calls to `fprintf`** write to valid open files, in ~70 s and ~50 MB per file handle on average. A few source changes were needed (hand-modelling one function, cloning functions where context-insensitive execution states lost precision).

Takeaways

- Property simulation = path sensitivity restricted to branches that matter for the property; polynomial, and precise in practice.
- Parametric in the execution-state domain (constant propagation chosen).
- Sound, but alias handling is coarse (weak updates via may-flow VFG) — the limitation that paper 3 addresses.

3. Fink, Yahav, Dor, Ramalingam, Geay — *Effective Typestate Verification in the Presence of Aliasing* (TOSEM 2008)

Goal

Sound typestate verification for real Java programs (e.g. "do not call `getInputStream()` on an unconnected `Socket`", "call `hasNext()` before `next()`", "do not read a closed stream"). In Java every object is on the heap and libraries add layers of encapsulation, so the central problem is **aliasing**. With only flow-insensitive may points-to information (SPA §11.1–11.4) a typestate transition through a pointer must be a **weak update** (the object may or may not have transitioned), which prevents verifying properties like `connect` → `getInputStream`. The paper develops abstractions that enable **strong updates** cheaply, and combines them in a **staged** verifier.

Setting

- Typestate property: DFA $\langle \Sigma, Q, \delta, init, Q \setminus \{err\} \rangle$ with err absorbing; observable operations are method calls.
- A preliminary flow-insensitive, (selectively) context-sensitive, Andersen-style points-to analysis gives abstract objects named by allocation site, called **instance keys** (SPA §11.1–11.2, §11.6).
- **Separation**: each abstract object is verified independently.
- A **point of potential failure** (PPF) is an operation that may lead to err; a pair (abstract object, PPF) is a verification problem; the set of unsolved pairs is the **verification scope**, passed from stage to stage.
- **Instrumented concrete semantics**: the concrete state $\langle L, \rho, h \rangle$ is extended with a map $typestate : L \to Q$; access paths $x.f_1.\ldots.f_k$ name objects. Soundness of abstractions is stated w.r.t. this semantics, in the abstract-interpretation style of SPA ch. 12.

Parametric abstract domain

An abstract state is a set of tuples

$$\langle o,\ unique,\ typestate,\ AP_{must},\ May,\ AP_{mustNot} \rangle$$

- o: instance key; *typestate*: its state;
- *unique*: at most one concrete object from this allocation site is live;
- AP_{must}: access paths that **must** point to the object;
- May: whether other access paths (not in AP_{must}) may point to it;
- $AP_{mustNot}$: access paths that definitely **do not** point to it.

Access-path sets are bounded in length (like k-limiting) and width, so the domain is finite. Must/must-not sets need not be complete, so information can always be dropped soundly (drop from $AP_{mustNot}$, or drop from AP_{must} and set $May = \text{true}$). Flow functions handle one tuple at a time, so they are distributive and the analysis runs on an **IFDS/RHS tabulation solver** (SPA §9.4) with the functional approach to context sensitivity.

Typestate transition at `e.op()` (simplified):

- if $e \in AP_{mustNot}$: no change (the object is not the receiver);
- if $e \in AP_{must}$, or *unique* and $pt(e) = \{o\}$: **strong update** (replace the typestate);
- otherwise: **weak update** (keep both old and new tuple).

The verifiers (increasing cost and precision)

1. **FI (flow-insensitive feasibility)**: using the set of operations that may be applied to each abstract object, check whether *err* is reachable in the automaton at all (e.g. a stream that is never closed cannot violate "read after close").
2. **LocalFocus (intraprocedural)**: flow-sensitive, one method at a time, unknown initial context, calls treated conservatively. Handles objects that do not escape their method (most iterators).
3. **Base**: interprocedural, tuples $\langle o, typestate \rangle$ only, all weak updates (comparable to ESP's alias handling).
4. **Unique** (*uniqueness analysis*): an allocation site emits $\langle o, true, init \rangle$ the first time; if a tuple for o reaches the same allocation site again, *unique* becomes false. A preliminary **liveness analysis** kills tuples of instance keys that cannot be live, so the common pattern "object dies before its allocation site runs again" and singletons stay unique. A unique object with a singleton points-to set gets strong updates.
5. **APMust**: adds must access paths (strong updates when the receiver expression is in AP_{must}).
6. **APFocus**: adds must-not paths and the **focus** operation, inspired by shape analysis (Sagiv–Reps–Wilhelm): when an operation `e.op()` would otherwise cause a weak update, split the tuple into one where $e \in AP_{must}$ (object pointed to by e, strong update) and one where $e \in AP_{mustNot}$ (all other objects, no update). This lets the analysis distinguish "the socket pointed to by `s`" from "the other sockets of the same allocation site", e.g. in a loop over a collection.
7. **Staged**: LocalFocus → Unique → APFocus, each stage running only on the scope left by the previous one (FI runs first in every verifier). Same precision as APFocus, usually faster.

Further techniques

- **Focus and polymorphism**: correlated dynamic dispatches (e.g. two `Iterator` implementations returned by `iterator()`) behave like correlated branches; must-not information lets call edges skip infeasible targets, giving a form of path sensitivity without explicit predicates (contrast with ESP).
- **Join / subsumption**: tuples are ordered by precision ($t_1 \sqsubseteq t_2$ when same object and typestate, and t_2 has less must/must-not/unique information). Propagating only the less precise tuple loses nothing in the final fixed point and avoids an exponential blowup of must-sets (one per path). The RHS solver is generalised to joins other than set union. A heuristic also discards old must-not paths on each focus.
- **Omission-closed properties** (valid sequences closed under deleting operations, e.g. `read* ; close`): strong updates using **may** information alone are sound, because any error sequence found with weak updates contains an error subsequence found with strong updates. Not valid for other classes (e.g. "repeatable enabling sequence" properties).
- **Sparsification**: prune the call graph to nodes that can reach methods writing variables relevant to the tracked objects (median supergraph size reduced ~50×); earlier stages further shrink the graph for later ones.

Implementation and results

Built on an Andersen-style, field-sensitive analysis over SSA with selective **object sensitivity** for collections and I/O streams, plus one-level call strings for a few factory methods. Unsound on concurrency, reflection (handled by tracking casts), native code (hand-written models) and excluded GUI libraries; otherwise sound.

On 17 benchmarks (up to ~290K bytecode statements including libraries) and 11 properties (`Iterator`, `InputStream`, `Socket`, `KeyStore`, `Signature`, `Stack`, `Vector`, ...), the share of PPFs verified:

FI	LocalFocus	Base	Unique	APMust	APFocus / Staged
30%	64%	68%	72%	85%	**93%**

Staging speeds up 9 of the 10 longest-running programs (median $-34-$85%). Most remaining warnings come from one idiom in SPECjvm98 (a `PrintStream` cached in a static field, 241 of 365) and from properties that cannot express the real invariant (e.g. a `Vector` accessed after checking `size() > 0`). Precision of the preliminary pointer analysis is critical: with a context-insensitive one, many runs time out.

Takeaways

- The key obstacle in typestate verification is aliasing; strong updates are essential.
- Cheap ways to get strong updates: uniqueness (+ liveness), must access paths, focus with must-not paths, and property structure (omission closure).
- Combined typestate+alias domain on top of an IFDS solver, with a staged pipeline of verifiers of increasing cost, scales to real Java programs.

Comparison

	metal/xgcc (1)	ESP (2)	Fink et al. (3)
Goal	find bugs	verify	verify
Soundness	unsound by design	sound	sound (modulo reflection, natives, concurrency)
Language	C	C	Java
Specification	metal SMs + patterns + C code	FSM + patterns	typestate DFA on method calls
Path sensitivity	DFS paths + cheap false-path pruning	property simulation	via must/must-not + focus
Aliasing	synonyms (local)	flow-insensitive VFG, weak updates	integrated must/must-not domain, strong updates
Interprocedural	top-down, function and suffix summaries	RHS, context-sensitive on property state	IFDS/RHS tabulation
Scaling trick	block caching, independence of tuples	merge by property state, one value at a time	separation, staging, sparsification